

Group Name:

TOCTOU (/tɒk 'tuːə/)

Group Number:

23

Group Members:

Student Number	Title	First Name	Last Name
23832656	Mr	Peter	Fang
23417131	Mx	Seb	Gazey
23804015	Mr	Alec	Hassell
23478063	Mr	Scotty	Maw
23926055	Lord	Henry	Yau

1. Team Communication & Responsibilities

1.1 Team Communication

The team had our first meeting on Wednesday (the 09/04/2025) at 12:00 (the group just so happened to all be available at 12-2pm on a Wednesday), where we decided to continue to meet at the same time each week, in person. If deemed necessary as the project progresses, we agreed to meet more often, but a minimum of once a week seemed like a suitable choice for maintaining the delicate balance between ease of tracking progress without impeding the Computer Science students very (very) busy social lives.

We quickly realised that our form of communication for outside of our meetings required decent support across different Operating Systems, while maintaining usability (ruling out IRC), and so our group settled on Discord as a suitable option.

1.2 Team Responsibilities

A main focus of the first meeting was working out the different skillsets of each group member, to allow for a more efficient allocation of tasks. We soon noted that the group had a varying level of technical skill, which helped with working out who worked on which parts of the second phase of the project.

That said, everyone will, of course, be expected to write code, and so a preliminary division of labour was allocated as so:

- **Peter & Henry:** *Admin and operations access*
- **Seb:** *Session management*
- **Alec:** *Role-based access control*
- **Scotty:** *Player authentication*

(Note that this is only the allocation of the coding related tasks, other responsibilities are discussed later)

The division of coding responsibilities was mostly decided as a group, with each member picking their preferred area. This division is subject to change, once the group has more information as to the requirements of the project, however it provides a nice starting point for our group to have an idea of what each person is working on.

1.3 Other Responsibilities

Of course, a group project wouldn't be complete without the addition of other requirements and responsibilities, and so some other tasks are listed below:

- **Peter:** Developing testing for the system, which will involve working closely with Alec and Scotty.
- **Henry:** Assisting with documentation, and ensuring tasks are carried out (and lending a hand to any tasks if needed).
- **Seb:** Management, coordinating team efforts, setting up version control system/linting, and documentation.
- **Alec:** Taking minutes at each meeting, threat modelling
- **Scotty:** Design, working on how secure coding principles can be used in the project

There are undoubtedly areas that we have missed in the above list that we will realise once we start phase 2 of the project, and so a critical element to our teams success is being able to adapt to new (or previously unrealised) requirements.

1.4 Progress Tracking and Accountability

Arguably the biggest part to a successful group project is effective progress tracking, and consistent accountability, and so we knew to focus on this at the first meeting.

We decided a solution to this was by having a group member who would be responsible for ensuring tasks aren't falling behind (Henry), and if they are, alerting the team, and being able to jump on said tasks (by having a reduced amount of "set" tasks) to assist with the tasks being completed on time.

A shared Google Sheets spreadsheet was also created, with the purpose to help Henry, and the group as a whole, keep track of the progress of each task.

Finally, meeting in person once a week was deemed the best way to keep accountability, as our group felt that a regular face-to-face meeting would help to remind us about who we would be letting down (as well as making it easier for Henry to reach out to any members who might need assistance with finishing tasks).

Of course, our group also understands that life can get very hectic, and the main request we had for each other is to let the other members know immediately if we start falling behind so that we can adjust tasks and priorities before it gets out of hand.

2. Version Control Strategy

Git was chosen as the underlying VCS, as even though there are (technically) better options, (see [Jujutsu](#), [Sapling](#), or [Pijul](#), along with the classics such as Fossil, Darcs, or Mercurial), due to Git's extreme popularity (and 4 out of 5 group members unwillingness to learn how to use a new SCM system while also working on the project), we decided to just use Git.

2.1 Remote Source Code Location

Our group decided that GitHub would be a suitable main online VCS provider to host our repository, mostly due to members familiarity with it (and BitBucket's dark-mode *still* being under beta testing).

GitHub also provides GitHub Actions, which one of our group members has experience with, which will be discussed later on in this report in [Part 3.2](#). We considered mirroring our repository to one of our members home-server, which has a simple Git based VCS server, however we decided against this, as we feel

confident that if GitHub goes down, we will have much bigger problems than this project.

However, if GitHub *does* go down, but the project is still our biggest concern, we will still have local copies on each of our devices, so providing that every group member does not suffer the loss of their devices, *and* GitHub also goes down *all at the same time*, we will then migrate the repository to the aforementioned group member's Git server (in which case, we will be using [Soft Serve](#) as our main VCS server)

2.2 Merging Strategies

Each group member's GitHub account was added to the repository and at the first meeting the version control fanatic discussed our planned approach, receiving the go ahead.

The overall system will be to use separate branches for each large addition to the project, however each group member is also allowed to have a *up to date* branch specifically for the addition of smaller changes, e.g. fixing typos in documentation etc. Each branch, when ready, will then have a pull request created to merge it into the main/dev branch (or any other branch being worked on by a separate member).

To mitigate against merge conflicts, a few strategies will be implemented. Regular communication (and a clear layout of responsibilities as listed above, see [1.2 Team Responsibilities](#)) help to ensure there are no major overlapping code changes. If this is unavoidable (for example, in the case of Peter and Henry), regular merging will be implemented. For features/branches that are worked on by a single person, regularly merging main into the branch is planned.

If merge conflicts *do* occur (likely more of a case of 'when' not 'if'), the team will work together to resolve them, seeking assistance from the person(s) most familiar with the conflicting code if possible.

In the absolute worse case, we plan to implement the xkcd/1597 [approach](#), however, we are all collectively hoping this will not be necessary.

2.3 Branching Systems

Since the group's main SCM enjoyer usually uses [Sapling](#), we did not really have any opinions on branching strategies, so our group kept it simple by having a main branch (creatively named "main"), and a "dev" branch which will be used for testing.

2.4 VCS Policies

The VCS policies our group decided on were not overly complex, however they were intentionally quite restrictive.

At least one review (and approval) is required before the branch can be merged, the branch being merged into the main and/or dev branch(es) *must* be up to date with the destination branch, and the GitHub Actions discussed below (see [3.2 Additional Tooling](#)) must also pass before the branch is able to be merged.

Commit messages should be kept to the recommended limit (of 72 characters), with a clear descriptions to be placed in the commit description. However, this will not be enforced, and group members are encouraged to prioritise clarity over brevity.

3. Development Tools

3.1 Editors

There will be three main editors in use for this project, Visual Studio Code (VS Code), Emacs and Vim.

The majority of the group plan to use VS Code for this project, as it is what they are used to and are comfortable using, and it has a wide plugin ecosystem. [VS Code Remote Development](#) and its SSH feature will be used by members who would like to use VS Code, still ensures compliance with the [CITS3007 standard development environment](#) (SDE). For other members working directly in the SDE, Vim will be used, for reasons including members' familiarity, and its lightweight nature making it an attractive option in a resource constrained VM.

The group discussed that whether using different editors would cause any issues, however the Emacs user was confident that they would be able to replicate any necessary features in Emacs (eg, Tramp instead of Remote Development), and if not will (begrudgingly) switch over to using VS Code for the remainder of the project.

3.2 Additional Tooling

This will be split up into 4 main sections - formatting, static analysis/linting, dynamic analysis, and the CI/CD implementation of these.

Note: The group was absent from the lecture covering Fuzzing (due to attending the first meeting for this project), and so implementing fuzzing has not been discussed

at a group meeting yet, this is planned to be addressed and implemented at the second meeting, which is the day that this report is due, and so the usage of fuzzing is not covered in this report, however it will almost definitely be used in the testing of this project.

3.2.1 Formatting

Consistent formatting is important to ensure a codebase remains readable, and so [Uncrustify](#) was setup for formatting all of our code. Somewhat controversially, the style used was derived from an Allman/BSD Style.

Due to the brackets being on a separate line to the control statement, commenting out a block of code/control statement while refactoring is less likely to cause errors via missing or dangling braces.

However, this is somewhat mitigated by the modern editors (and Emacs) being used by the group, so the real reason for why this style was used is that the group member who setup the CI/CD for the project prefers Allman style indentation, and no-one felt strongly enough about that member's questionable stylistic choices to change it.

Plugins for Uncrustify exist for both VS Code and Emacs, however in case these were not used, more methods for ensuring formatting are explained below

3.2.2 Static Analysis/Linting

While Linters and Static Analysis are different tools, they have some cross over, with some tools being marked as both. Hence, both will be discussed here.

Our strategy of choosing tooling was very much based around “how many tools can we use”, and so we used as many as possible (while ensuring the group understood how to use them effectively).

3.2.2.1 Linters:

- **clang-tidy**

- While using compiler flags such as `gcc -Wall` provide linting (and are still a crucial part of writing secure code), clang-tidy was chosen due to its deeper (and more modern) analysis.
- Clang-tidy's ability to analyse for undefined behaviour provides much more in depth warnings of unsafe code, increasing the safety of our code.
- It also does a nice job of fixing errors where it can, which saves a considerable amount of time.

- The flags used by our group are `--fix-errors -warnings-as-errors=*`

- **cpplint**

- While this might seem like an odd choice when using Uncrustify, cpluspluslint also enforces function size limits and code aware indentation, which can prevent hidden logic bugs.
- It also flags style violations, unlike Uncrustify, which our team finds useful.
- There are a lot of flags being used here to prevent conflicting with Uncrustify, however the main focus is on the `+runtime` and `+build` filters, which targetted things that Uncrustify doesn't

3.2.2.2 Static Analysis Tools:

- **cppcheck**

- This was chosen as it was a lot easier for our group to setup and use effectively than something like Splint, while still providing an effective analysis to warn of common issues that aren't always picked up by compilers.
- The flag used is `--check-level=exhaustive`, however this is mostly for pre-commit setup (more on that later).

- **flawfinder**

- Flawfinder will be very helpful for explaining the context of any issues, which helps us quickly determine the importance of a problem.
- It provides a very useful first line of checking where problems may be (such as using unsafe functions), before using heavier static tools for deeper auditing.

Finally, as it is worth mentioning here before going into more detail of the process later, we have a vast number of compiler flags and warnings designed to help us be aware of any issues we may face before running linters and static analysis tools.

3.2.3 Dynamic Analysis

We implemented a few different tools for this, however our main approach was to use Sanitizers and Valgrind.

3.2.3.1 Sanitizers

We plan to use as many Google Sanitizers as we can, as in our environment we can control the compilation flags. These will provide us with valuable abilities to do dynamic analysis on our programs, vastly increasing its safety.

A list of the current sanitisers in use (over different compilation commands as some are not compatible with each other) is listed below, however when we start writing code this number will probably increase:

undefined, float-divide-by-zero, float-cast-overflow, bounds, null, return, vptr, undefined, address, leak, thread, memory

3.2.3.2 Valgrind

Our group will possibly use Valgrind as well, although like with the note about fuzzing above, we are yet to discuss if/how we will use it (we only have sanitisers setup because the group member doing CI/CD has used them before). Hence, we are unsure how/if we will use this, although it is a strong possibility due to its flexibility in debugging (for example if we cannot build a library we use with the sanitisers we could use Valgrind to assist us).

3.2.4 CI/CD

The above tools are only useful if they are *used*, and so to ensure usage, our team setup a few systems to make the tools as easy to use as possible. These are split into two main parts, Pre-Commits, and GitHub Actions.

3.2.4.1 Pre-Commit Hooks

We decided to make heavy usage of [pre-commit](#) hooks to ensure formatting, linters, and analysers were run to prevent “dirty” commits from being added to branches.

Formatting before commits are made will help with any merges that need to be done, as it ensures brackets/indentation (and overall code styling) was consistent, reducing the likelihood at any unnecessary merge conflicts, which will reduce the strain on working on code collaboratively.

Hence, we set up a simple Uncrustify hook here to check if formatting has been done to all files. We also ran clang-tidy, cppcheck, cpplint and flawfinder, preventing any commits being made if insecure code/issues were found by these tools (and in the case of clang-tidy, fixing any issues that it could automatically).

Finally, to make sure our code compiled with all of our Makefile options (with different compiler flags), we ran a simple script that would run [bear](#) on our Makefile, generating a `compile_commands.json` file (which is also used by clang-tidy), and attempting to build each option in our Makefile. By running these checks locally before pushing to the remote (and using only GitHub actions), it saves our group time, because instead of having to do multiple pushes to the

remote (one to find any issues and then another fixing those issues), we will be alerted to any issues before they left our local machines.

3.2.4.2 GitHub Actions

That is, of course, not to say that GitHub Actions do not provide a useful service for our development system. The group have two main GitHub Actions setup, one to ensure formatting had been done, and another to run flawfinder on the code.

Any pull request made triggers an action that runs Uncrustify on our code, and if any files were found that did not meet the agreed style standards, the action fails, blocking that Pull Request from being able to merge until the formatting is been fixed. This also has the benefit of alerting our group members to a faulty pre-commit setup, which means we would need to make sure if the linting/analysis tools were also skipped (along with ensuring the aforementioned benefits that come with formatted code).

The second action simply runs an aggressive flawfinder on the pull request, and if any issues were found it would return the issues in a more readable formatted comment on the pull request, which helps the reviewers with spotting potential issues quickly. Because we are running a very aggressive search with flawfinder, this action does not prevent merging on errors (unlike the Uncrustify action), instead it is more aimed at being a tool that the code reviewers could use to help them check for potential flaws.

4. Key Secure Coding Practices for Phase 2

While we will be implementing many different security-related tools and practices in phase 2 of this project, three such tools/practices (that have not already been discussed above) that we will be using are:

4.1 Consistent Implementation of Security Tooling and Decisions Throughout the SDLC

4.1.1 Relevancy:

Security must be a consideration from the beginning of a project, because trying to add in security at the end of the project is difficult and unsafe. In a system like an ACS, which has to deal with authentication, session handling and privileged access, if we do not consider security from the beginning (such as design and

development), our system will very quickly end up with large architectural vulnerabilities, which would require a large amount of time and work to fix.

4.1.2 How it will be applied:

Security considerations will be integrated into each stage of the project:

- **Planning & Design:** Threat modeling (such as STRIDE) will be used to identify and mitigate key risks before code is written.
- **Implementation:** Secure coding practices (such as input validation), linters and static analysis tools will be enforced from day one (see [3.2.2 Static Analysis/Linting](#) for more information about the specific tools we will be using).
- **Testing:** Runtime tools, such as the sanitisers, as discussed in [3.2.3.1 Sanitizers](#), and Valgrind (see [3.2.3.2 Valgrind](#), and static analysis tools (see [3.2.2 Static Analysis/Linting](#)) will be part of both manual and CI testing (see [3.2.4 CI/CD](#)).
- **Code Review:** All pull requests will be reviewed for adherence to secure coding practices (as discussed in [5.2 Maintenance of Code Quality](#), [2.4 VCS Policies](#), and [3.2.4.2 GitHub Actions](#)).

4.1.3 How the group will ensure it is effectively used:

Meeting minutes have a section for any security-related decisions made and after each meeting these get added into a document that is shared among the team, so that each team member is aware of each decision, which will help with not only writing code, but also remaining consistent when reviewing code. CI hooks have been configured to run formatters, linters, sanitizers, and static analysis tools on each commit, which helps with the review of pull requests made, reducing the effort required by the team to use these tools. Security is a large part of the code review procedure, because we will not just be reviewing functionality.

4.2 Input Validation & Bounds Checking

4.2.1 Relevancy:

As the ACS will be handling user inputs (such as usernames, passwords, and session tokens), great care must be taken to parse (not only validating) inputs, to prevent buffer overflows and injection vulnerabilities. Further, we can only speculate on how our users will affect our gamestate, and what input channels they will have. We believe that bounds checking will be especially relevant to in-game resources, e.g. ensuring the most obscenely rich plutocrats don't experience

an underflow into a negative net-worth. There will undoubtedly be lots of channels of user input that will come up, on top of the above, potentially including movement inputs, marketplace-like menus and potentially player-to-player communications. Each avenue will need to be relevantly neutralised to avoid downstream components introducing unexpected and potentially harmful behaviour.

4.2.2 How it will be applied:

Multiple techniques will be used to ensure safe inputs, these include validating and parsing the input, but also sanitising and canonicalisation (normalisation) where necessary. All functions that deal with user input will also use safe C functions (such as `fgets` instead of `gets` and `snprintf` instead of `sprintf`), and will explicitly check lengths and formats before processing.

A similar technique will be to ensure environment variables are sanitised, because the ACS will likely need to be run with root privileges.

4.2.3 How the group will ensure it is effectively used:

Code reviews will be explicitly including input handling, and the `flawfinder` GitHub Action has been setup to help the reviewers find unsafe functions (however the reviewers will not solely be relying on `flawfinder`). The pre-commit setup has a large amount of linters and static analysers that will also pick up on unsafe functions before they can be committed to the working branch.

The group will also be testing sanitisation of inputs using fuzzing, however see the note in [3.2 Additional Tooling](#) as to why this is not discussed in much depth in this report.

4.3 Authentication & Credential Management

4.3.1 Relevancy:

A system like an ACS is directly responsible for managing who can access what, so naturally the authentication mechanism is what makes up the bulk of the system's security. Improper authentication can lead to unauthorized access, privilege escalation, or complete system compromise. Since C does not have built-in memory safety features, mishandling passwords or session tokens can also lead to vulnerabilities such as buffer overflows, memory leaks, or inadvertent exposure of secrets in memory. User's accounts will have huge variability in privileges and attributes – and the central idea of the game is role-playing a nightmarish neo-capitalist society. To remain faithful to these axioms, strict policing of accounts is a necessity. Therefore ensuring we can reliably moderate access to accounts by authenticating users, will be an important feature of our ACS.

4.3.2 How it will be applied:

Authentication in the ACS will be based on username-password pairs, securely stored and verified using cryptographic hashing. Specifically:

- Passwords will be hashed using Argon2 via a C-compatible library such as libargon2.
- A salt will be randomly generated and stored as a prefix of the hashed password to prevent precomputed dictionary attacks (e.g., rainbow tables).
- Memory used to store sensitive information (e.g., raw passwords) will be immediately cleared (`explicit_bzero` or safer options) after use to avoid leaving secrets in memory.

The ACS will also use:

- Account lockout after a configurable number of failed attempts
- Secure comparison functions (`consttime_memcmp`) to prevent timing attacks
- Session tracking with unique, securely generated session tokens (e.g., using `/dev/urandom`)
- Enforcing access control through a ban list, checked during authentication. Depending on what representation of users we are given in phase 2, this list could be limited to username-password pairs, or include IP addresses or other identifiers.

4.3.3 How the group will ensure it is effectively used:

The group will audit all code paths that deal with credential input, storage, and verification as part of the code review process. To enforce safe coding standards, static analysis tools (such as `flawfinder` and `cppcheck` as mentioned before) and linters will flag unsafe handling of user input or improper use of memory.

Test cases will be written to simulate common attack scenarios such as brute force login attempts, malformed credential inputs, and memory scraping. These tests will be included in CI workflows to catch regressions.

Additionally, a secure coding guideline for authentication will be documented and shared among the team to ensure consistent implementation practices. Periodic team reviews will evaluate whether authentication remains effective as new features are added.

5. Risk Management & Quality Assurance

5.1 Potential Risks & Mitigation Strategies

Risk	Impact	Likelihood	Mitigation Strategy
Team member illness or unavailability	Medium	Medium	Regular updates to the spreadsheet so responsibilities can be reassigned easily, along with clear and fast communication between team members if unavailability occurs.
Sudden death of a team member	Medium	Medium	We discovered that one of our team members rides a motorcycle to university, which drastically increases the likelihood of this event, and so the main mitigation taken was to ask them to <i>please</i> be careful and to do regular pushes to the remote repository.
Service outage (GitHub)	Low	Low	Each member kept local backups of the Git repository and tools used. However, as mentioned above , our group feels that if these systems go down we will have bigger issues.
Member loss of GitHub, Discord accounts	Medium	Medium	We all have each others student numbers, and so we can contact each other via email if we need a new account added to the discord group or the repository. This is quite a possible risk, as we have already had one member have their GitHub account banned and needed to have a new account added to the repository, and our team used this mitigation strategy to quickly solve this issue
Merge conflicts or broken main branch	High	Medium	Using a strict Git workflow and requiring passing checks before merging into main as discussed above

Risk	Impact	Likelihood	Mitigation Strategy
			in 2. Version Control System and 3.2 Additional Tooling
Security vulnerabilities in code	High	Medium	Using static analysis tools and include security as a focus during peer code review as discussed in 3.2 Additional Tooling .
Missed deadlines	High	Medium	Weekly meetings and a shared spreadsheet with deadlines along with starting early and assigning buffer time for unexpected issues will be used to (hopefully) mitigate this risk

5.2 Maintenance of Code Quality

Our group plans to follow the [SEI CERT C Coding Standard](#) to help ensure the security and quality of our code. As discussed in [2. Version Control Strategy](#), and [3.2 Additional Tooling](#) (in much more depth), a combination of peer reviews, automated testing, formatting, static analysis tools, linters, and dynamic analysis tools have will be used, with our team likely to add in more tools as we progress. Justifications for, and the reasoning behind, the tooling chosen is available at the sections linked above.

6. Group name

Our group name is TOCTOU.